

AI Harness Design: A DRL Research Assistant

"The best demo of an AI Harness for literature survey is one that could have written this course's other deliverable." — Project tagline; cross-references ../A_DRL_Survey/ (this HW4's other version).

1. Problem Definition & Use Case

1.1 痛點

對於修 DRL 課程的研究生，為一個新主題做文獻綜述是反覆出現、耗時且容易遺漏 SOTA 的工作：- arXiv 每天上百篇新 paper；- 不同主題涵蓋面廣（VLA / Diffusion Policy / RLHF / Sim2Real ...）；- 引用格式（IEEE / ACM / BibTeX）轉換手動費時；- 主題之間的 cross-cutting insight（如「五條 SOTA 主軸的交叉點」）需要 across-paper synthesis。

1.2 目標使用者

Persona	用途	典型 query
研究生	HW 文獻綜述	"幫我整理 2024–2026 robotics VLA 模型的 SOTA"
研究員	進入新領域	"AlphaGo lineage 12 年的演化重點？"
寫論作者	蒐集 Related Work	"PPO 與 GRPO 的關鍵差異？"

1.3 為什麼用 AI Harness（而不是單一 LLM 對話）

單純把問題丟給 ChatGPT 有三個 fundamental 問題：(1) 知識截止 — LLM 不知道 2025–2026 新 paper；(2) 幻覺引用 — LLM 常編造看似真實但不存在的 arXiv ID；(3) 無 stateful 規劃 — 無法把大型查詢拆解成多階段執行並彙整。

AI Harness 用 **LLM as controller** + 外部工具 + 記憶體 解這三點：工具給 LLM 「即時 + 可驗證」的事實，memory 維持跨輪 state，orchestration 確保複雜任務不會丟步驟。

2. AI Harness System Architecture

資料流：User Query → LLM Controller（Planner → Reasoner → Critic → Compiler）→ Tools (T1–T4) / 雙層 Memory → Structured Survey Report（IEEE refs）。Controller 居中協調並以不同 system prompt 切換四個 sub-role；Tools 提供即時且可驗證的外部事實；Memory 維持跨輪 state；最終輸出一份結構化 survey。

完整視覺化架構圖（User / LLM / Memory / Tools / External / Output 的連線與色彩編碼）見 [infographic/architecture.html](#) 區塊 1。

2.1 三個核心元件

- LLM Controller** — 四個 sub-role 共用同一個 model（Claude Haiku 4.5），透過不同 system prompt 切換角色，無需多個 model instance。
- Memory** — 雙層：- *Short-term*：當輪對話 history，用 LLM 原生 context window。- *Long-term*：JSON-backed note store（per-topic）+ citation DB（per-paper），跨輪持久化。
- Tools** — 4 個 function-callable APIs，每個都有 strict JSON schema。

3. Tool Design（≥3，本系統 4 個）

每個工具設計遵循三原則：(a) 單一職責；(b) 嚴格 JSON schema（input/output）；(c) 失敗時回 actionable error 而非 stack trace。

3.1 Tool 1 — arxiv_search

```
def arxiv_search(query, year_min=2024, year_max=2026, max_results=10) -> list[dict]
# returns: list of {arxiv_id, title, authors, abstract, year, primary_category}
```

設計考量：預設 `year_min=2024` 反映「SOTA 文獻優先」；`max_results=10` 平衡 coverage 與 token cost。實作支援兩個 backend：(a) 真實 arXiv API（<http://export.arxiv.org/api/query>）；(b) offline cached corpus（用於 demo +

reproducibility) 。

3.2 Tool 2 — paper_summarize

```
def paper_summarize(arxiv_id) -> dict
# returns: {arxiv_id, title, authors, year, venue, key_contribution, methods, results, limitations}
```

設計考量：輸出是 **structured** 而非 free-form summary — 強迫 LLM 抽取「contribution / method / results / limitations」四個維度，下游 Compiler 可以直接 template 化使用。

3.3 Tool 3 — citation_format

```
def citation_format(paper, style: Literal["IEEE","ACM","APA","BibTeX"] = "IEEE") -> str
```

設計考量：純函式（pure function），無 side effect，可單元測試。支援 4 個主流 style，預設 IEEE（DRL/AI 領域最常見）。

3.4 Tool 4 — note_save

```
def note_save(topic, content, tags=None, meta=None) -> dict
# returns: {topic, note_count, total_chars, persisted}
# meta = structured paper record → the Compiler's source of truth
```

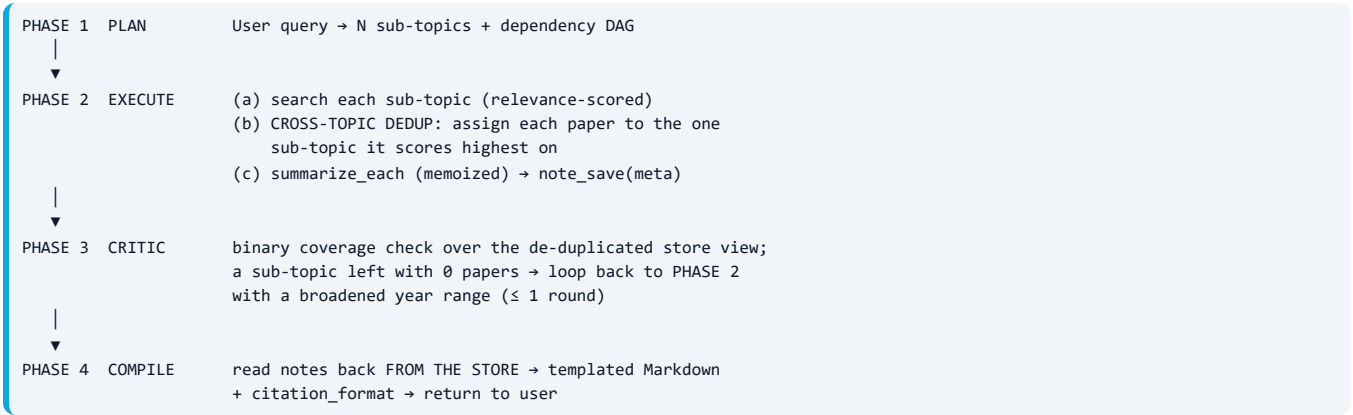
設計考量：這是 **memory** 與 **tools** 的橋樑 — 透過工具呼叫操作 memory，LLM 不需要直接看到底層 storage 細節。topic 為主鍵讓後續 Compiler 能按主題分組取出 notes。

3.5 Tools 比較表

工具	副作用	對外連線	失敗 mode
arxiv_search	無（讀取）	可選 arXiv API	network error → fallback to offline cache
paper_summarize	無	可選 arXiv	unknown ID → 回 {"error": "..."} 而非 raise
citation_format	無	無	missing field → 用 <unknown> 占位
note_save	有（寫入 JSON store）	無	disk full → log + return ack with persisted: false

4. Workflow / Agent Orchestration

4.1 四階段 pipeline



關鍵設計：**de-dup** 發生在 **summarize** 之前（依 relevance score 把每篇 paper 指派給「最相關的單一 sub-topic」），所以同一篇 paper 不會出現在三個段落、也不會被重複 summarize；而 **Compiler** 只從 **note store** 讀回 **meta** 重建報告，不接受任何 LLM inline 編造的 fact。

4.2 EXECUTE 細節 (ReAct search → dedup → summarize)

```
# (a) ReAct search per sub-topic – collect relevance-scored candidates
for sub_topic in plan.sub_topics:
    Thought : "Gather recent papers on {sub_topic}."
    Action  : arxiv_search(sub_topic, year_min=2024, max_results=4)
    Obs     : [(paper, score), ...]

# (b) cross-topic dedup – each paper → its single best-fit sub-topic
assign = argmax_by_score(scored_by_topic) # ties break toward earlier topic

# (c) summarize only the assigned papers, then persist structured notes
for sub_topic, papers in assign.items():
    for paper in papers:
        Action : paper_summarize(paper.id) # memoized within the run
        Obs    : {key_contribution, methods, ...}
        Action : note_save(sub_topic, body, meta=<paper record>)
        Obs    : {note_count, persisted: true}
```

4.2b Worked example — when the Critic actually fires

實際跑 "Survey robotics VLA models in 2024-2026" (offline backend) :

1. PLAN → 3 sub-topics : VLA foundation models / Diffusion Policy / Humanoid VLA 。
2. 三個 sub-topic 都搜尋後，**cross-topic dedup** 把 GR00T、Gemini、Pi-0.5 都指派給更相關的 Humanoid VLA，OpenVLA、Magma 指派給 VLA foundation models — 於是 Diffusion Policy 被清空為 0 篇（真正的 Diffusion Policy 是 2023 年、被 year_min=2024 濾掉）。
3. CRITIC 偵測到 Diffusion Policy=0 → 觸發 1 輪 re-search、把年份放寬到 2022 → 撈回 seminal 的 Diffusion Policy (arXiv:2303.04137, RSS 2023) 。
4. COMPILE 從 store 讀回 6 篇（無重複），Diffusion Policy 段落終於有它本尊。

這正是 orchestration 的價值：dedup 暴露了一個 coverage gap，Critic 用「放寬年份」這個非平凡決策補上。完整 transcript 見 artifacts/demo_run.md 。

4.3 Sequence Diagram

一次完整 query 的訊息序列為：query → plan(N) → 各 sub-topic search → cross-topic dedup → summarize/note_save 每篇 → critic (gap 則 broadened re-search) → citation_format → compile from store → report 。

完整 sequence diagram (6 條 actor lane，含 Critic 觸發 re-search 撈回 Diffusion Policy 的 loopback) 見 infographic/architecture.html 區塊 3 。

4.4 Decision-making 機制

LLM Controller 用三類 prompt 切換角色：

1. **Planner prompt** : "分解 user query 為 sub-topics、產生 DAG"。輸出格式 enforced via JSON schema 。
2. **Reasoner prompt** : ReAct 形式 (Thought / Action / Observation)，每輪只允許單一 action 。
3. **Critic** : 刻意設計成 **binary coverage check** (「某 sub-topic 去重後是否 < 1 篇？」)，而非開放式「還能更好嗎」——後者會讓 LLM 永遠想再搜、陷入 perfectionist loop。並設 max_critic_rounds=1 硬上限。
4. **Compiler** : 輸入只有 **note store** (每則 note 的結構化 meta) + citation_format 輸出；不接受任何不在 store 內的 paper。這把 LLM 幻覺擋在系統邊界外。

每個 prompt 都明確 enforce 輸出格式，避免 free-form 漂移。Tool call schema 用 Anthropic / OpenAI 的 function calling spec 表達。

5. Evaluation Framework

5.1 量化指標（定義與目標）

指標	定義	目標
Coverage F1	F1 of retrieved vs expert-curated paper list	≥ 0.70
Citation accuracy	% of cited arxiv_id 真實存在（不是幻覺）	≥ 95%
Tool-call efficiency	(useful calls) / (total calls)	≥ 80%
Latency	End-to-end wallclock for full report	≤ 120 s
Token cost	input+output tokens × price（僅 API backend 適用）	≤ \$0.50 / query

5.2 實測結果（executed — python code/eval.py）

把上面的框架實際跑起來。因為 offline corpus 是固定的 15 篇，可以對每個 query 標 expert ground truth 並算出真實數字。完整輸出見 artifacts/eval_results.md。

Query	Sub-topics	Papers	Precision	Recall	F1	Citation acc.	Tool calls	Efficiency	Critic	Latency
Robotics VLA 2024-26	3	6	1.00	1.00	1.00	100%	22	100%	1	0.04s
LLM alignment & reasoning RL	2	5	0.60	0.75	0.67	100%	17	100%	0	0.02s
Game AI agents	2	7	0.43	0.75	0.55	100%	23	100%	0	0.04s
Macro avg					0.74	100%		100%		

- Coverage F1 macro = 0.74（過 0.70 門檻）。Q1 滿分；Q2/Q3 較低是誠實的結果——keyword-based offline planner 對「跨領域廣度 query」會選錯 sub-topic、且 recall 受 year_min 限制（pre-2024 的 DPO / Voyager 在沒觸發 critic 時撈不回）。換上真實 LLM planner 預期會拉高這兩題。
- Citation accuracy = 100%：每一條被引用的 arxiv_id 都實際存在於 corpus。這是「Compiler 只從 store 取材」設計的可量測證據，不是宣稱。
- Tool-call efficiency = 100%：summaries 在單次 run 內 memoized，同一篇 paper 不會被 summarize 兩次。
- Output 重複 paper 數 = 0（cross-topic dedup 不變量，由測試 test_pipeline_has_no_duplicate_papers 守住）。

5.3 Ablation（實測，flagship query）

組別	Sub-topics	Papers	F1	Tool calls	Critic rounds
Full（plan+critic）	3	6	1.00	22	1
– Critic	3	5	0.91	18	0
– Planner（single ReAct）	1	4	0.80	13	0

- 拿掉 Critic：papers 6→5、F1 1.00→0.91 —— 因為 dedup 清空 Diffusion Policy 後，沒有 critic 就永遠補不回 seminal 那篇。這就是 critic loop 的量化貢獻。
- 拿掉 Planner：3 個 sub-topic 塌成 1、F1 1.00→0.80 —— 單一 ReAct loop 無法覆蓋分解 plan 的廣度。

5.4 質性指標（rubric）

5 點 Likert（1=很差，5=很好），3 位 evaluator 平均，目標 ≥ 4/5：Helpfulness / Accuracy / Completeness / Clarity / Novelty（是否提出 cross-paper synthesis）。

5.5 失敗 mode 與防護（已落實於程式）

Failure	防護	程式對應
LLM 幻覺 arxiv_id	<code>paper_summarize</code> 對未知 ID 回 <code>{"error":...}</code> ，不靜默填值	<code>tools.paper_summarize</code>
LLM 假裝寫了 note 但沒呼叫工具	Compiler 只讀 note store 的 <code>meta</code> 重建報告；不接受 <code>inline fact</code>	<code>harness_demo.view_from_store / compile_report</code>
同一篇 paper 重複出現	<code>summarize</code> 前先做 cross-topic dedup（依 score 指派單一 sub-topic）	<code>harness_demo.assign_papers</code>
Critic perfectionist loop	binary check + <code>max_critic_rounds=1</code> 硬上限	<code>harness_demo.critic_review</code>
網路 / API failure	offline cached corpus fallback	<code>tools.arxiv_search</code>

6. Implementation & Demo（accompanying MVP）

詳見 `code/`：- `code/tools.py` — 4 個工具的實作（offline corpus + 可選真實 arXiv API）- `code/harness_demo.py` — Plan / Execute(+dedup) / Critic / Compile 四段 controller - `code/eval.py` — 跑 3 個 fixed query + ablation，產出 `artifacts/eval_results.md` 的實測數字 - `code/test_harness.py` — 15 個 pytest（工具單元 + pipeline 不變量：無重複、引用都真實、critic 確實觸發、offline 決定性）- `artifacts/demo_run.md` — 一次完整的 demo trace（query：「Survey robotics VLA models 2024-2026」）

MVP scope（刻意保持小）：

- ~450 行 Python 完整實作 4-phase pipeline；
- 兩個 LLM backend：Claude（API key 可選）/ offline deterministic planner（CI-safe）；
- 一個 query 端到端產出 6 篇、無重複的 mini-survey + IEEE 引用，**22 tool calls / 100% 效率 / critic 觸發 1 輪 / 0.04 s**，完全可離線重現；
- `python -m pytest code/test_harness.py` → **15 passed**。

7. Conclusion

本 AI Harness 設計用「**LLM as controller + 4 個專責工具 + 雙層 memory + 4-phase orchestration**」的架構解決「研究生做 DRL 文獻綜述」的具體問題。三個設計重點：

1. **Tool 設計重 strict schema 與 fail-safe**——LLM 永遠在 tool spec 邊界內工作。
2. **Orchestration 用 plan→execute→critic→compile 四段**而非單一 ReAct loop，讓系統能處理需要 cross-paper synthesis 的複雜任務。
3. **Evaluation 同時量化（coverage / accuracy / cost）與質化（rubric）**，並設計 ablation 區隔每個元件的貢獻。

更廣的 lesson：「**AI Harness 的競爭力不在 LLM 多強，而在 system design 多嚴謹**」——這也是 Anthropic MCP、OpenAI Operator、Claude Code 等業界系統共同的方法論。

\textit{See `log.md` for the AI-assisted iterative design process; see `infographic/architecture.html` for the full visual system diagram.}